Save as:

`Java_Level_4_Exceptions_Generics.md`

---

# Java Level 4 — Exception Handling & Generics

## Overview

Level 4 focuses on two important areas of Core Java:

1. **Exception Handling** — writing robust code that can handle failures gracefully.
2. **Generics** — writing reusable, type-safe Java code.

By the end of this level, you should be comfortable with:

- `try`
- `catch`
- `finally`
- Checked Exceptions
- Unchecked Exceptions
- `throw`
- `throws`
- Custom Exceptions
- Exception Propagation
- `try-with-resources`
- Exception Handling Best Practices
- Generic Classes
- Generic Methods
- Generic Interfaces
- Multiple Type Parameters
- Bounded Generics
- `? extends`
- `? super`
- PECS
- Practical Generic Classes

---

# Part 1 — Exception Handling

## 1. What is an Exception?

An exception is an event that interrupts the normal execution of a program.

Example:

```
int result = 10 / 0;
```

This causes:

```
ArithmeticException
```

Instead of allowing the program to terminate unexpectedly, we can handle the exception.

---

# 2. `try` / `catch`

Basic syntax:

```
try {

    // risky code

} catch (ExceptionType e) {

    // handle exception

}
```

Example:

```
try {

    int result = 10 / 0;

} catch (ArithmeticException e) {

    System.out.println(
            "Cannot divide by zero"
    );
}
```

—

Flow:

```
try
 ↓
Exception occurs
 ↓
catch
 ↓
Program continues
```

---

# 3. Multiple `catch` Blocks

Different exceptions can be handled separately.

```
try {

    int[] numbers = {10, 20, 30};

    System.out.println(
            numbers[5]
    );

} catch (ArithmeticException e) {

    System.out.println(
            "Arithmetic error"
    );

} catch (ArrayIndexOutOfBoundsException e) {

    System.out.println(
            "Invalid array index"
    );
}
```

Java executes the first compatible `catch`.

---

# 4. Catching Multiple Exceptions

If different exceptions require the same handling:

```
try {

    // code
```

```
    // code

} catch (
        ArithmeticException |
        NumberFormatException e
) {

    System.out.println(
            "Invalid input"
    );
}
```

This is called **multi-catch**.

---

# 5. `finally`

`finally` is used for code that should execute after `try` / `catch`.

```
try {

    System.out.println("Try");

} catch (Exception e) {

    System.out.println("Catch");

} finally {

    System.out.println("Finally");
}
```

The `finally` block normally executes whether an exception occurs or not.

Common use:

```
Cleanup
Close resources
Release resources
```

---

# 6. Checked Exceptions

Checked exceptions are exceptions that the compiler requires you to handle or declare.

Example:

```
FileReader reader =
        new FileReader("data.txt");
```

This can throw:

```
FileNotFoundException
```

You must either use:

```
try/catch
```

or:

```
throws
```

Example:

```
public void readFile()
        throws IOException {

    FileReader reader =
            new FileReader("data.txt");
}
```

Common checked exceptions:

```
IOException
SQLException
FileNotFoundException
```

---

# 7. Unchecked Exceptions

Unchecked exceptions occur at runtime and don't need to be explicitly declared.

They generally extend:

```
RuntimeException
```

Examples:

```
ArithmeticException
NullPointerException
ArrayIndexOutOfBoundsException
IllegalArgumentException
NumberFormatException
```

Example:

```
String value = null;

System.out.println(
        value.length()
);
```

This causes:

```
NullPointerException
```

---

# 8. Checked vs Unchecked

| Checked | Unchecked |
| --- | --- |
| Checked by compiler | Occurs at runtime |
| Must catch or declare | No requirement to catch/declare |
| Usually external/recoverable failures | Usually programming/validation errors |
| `IOException` | `NullPointerException` |
| `SQLException` | `IllegalArgumentException` |

Mental model:

```
Checked
    ↓
Compiler forces handling

Unchecked
    ↓
Runtime problem
```

# 9. `throw`

`throw` is used to **actually throw an exception**.

```
public void withdraw(double amount) {

    if (amount <= 0) {

        throw new IllegalArgumentException(
                "Amount must be positive"
        );
    }
}
```

Syntax:

```
throw new ExceptionType("message");
```

# 10. `throws`

`throws` declares that a method **may throw an exception**.

```
public void readFile()
        throws IOException {

    // file operations
}
```

It does not actually throw the exception.

### Difference

```
throw
 ↓
Actually throws exception

throws
 ↓
Declares possible exception
```

Example:

```
throw new IOException();
```

versus:

```
void readFile() throws IOException
```

---

# 11. Custom Exceptions

You can create your own exception.

Example:

```
class UserNotFoundException
        extends RuntimeException {

    public UserNotFoundException(
            String message
    ) {
        super(message);
    }
}
```

Use it:

```
public User findUser(long id) {

    User user = findFromDatabase(id);

    if (user == null) {

        throw new UserNotFoundException(
                "User not found: " + id
        );
    }

    return user;
}
```

This is extremely common in backend applications.

Examples:

```
UserNotFoundException
OrderNotFoundException
InsufficientBalanceException
InvalidPaymentException
```

---

# 12. Exception Propagation

Exception propagation means an unhandled exception moves **up the method call stack** until it is handled.

Example:

```
static void methodC() {

    int result = 10 / 0;
}

static void methodB() {

    methodC();
}

static void methodA() {

    methodB();
}
```

If `main()` calls:

```
try {

    methodA();

} catch (ArithmeticException e) {

    System.out.println(
            "Exception handled"
    );
}
```

The flow is:

```
main()
 ↓
methodA()
```

```
 ↓
methodB()
 ↓
methodC()
 ↓
Exception
 ↓
methodB() ❌ not handled
 ↓
methodA() ❌ not handled
 ↓
main() ✅ handled
```

The first compatible `catch` stops propagation.

---

# 13. Exception Propagation in Backend Applications

A typical backend structure:

```
Controller
    ↓
Service
    ↓
Repository
    ↓
Database
```

Suppose the repository throws:

```
throw new UserNotFoundException(
        "User not found"
);
```

It can propagate:

```
Repository
    ↓
Service
    ↓
Controller
    ↓
Exception Handler
```

Later, in Spring Boot, we can use a global exception handler to convert this into:

```
UserNotFoundException
        ↓
HTTP 404
```

This is why understanding propagation is important.

---

# 14. `try-with-resources`

`try-with-resources` automatically closes resources.

Traditional approach:

```
FileReader reader = null;

try {

    reader = new FileReader("data.txt");

} finally {

    if (reader != null) {
        reader.close();
    }
}
```

Modern approach:

```
try (
    FileReader reader =
            new FileReader("data.txt")
) {

    // use reader

} catch (IOException e) {

    System.out.println(
            "File error"
    );
}
```

Java automatically closes the resource.

The resource must implement:

```
AutoCloseable
```

or:

```
Closeable
```

---

# 15. Multiple Resources

```
try (
    FileReader reader =
            new FileReader("input.txt");

    BufferedReader bufferedReader =
            new BufferedReader(reader)
) {

    System.out.println(
            bufferedReader.readLine()
    );

} catch (IOException e) {

    System.out.println(
            "File error"
    );
}
```

This becomes especially important later with JDBC:

```
Connection
Statement
ResultSet
```

---

# 16. Exception Handling Best Practices

### Catch specific exceptions

Prefer:

```
catch (UserNotFoundException e)
```

instead of:

```
catch (Exception e)
```

---

## Don't swallow exceptions

Avoid:

```
try {

    // code

} catch (Exception e) {

}
```

This makes debugging difficult.

---

## Preserve the original cause

```
catch (SQLException e) {

    throw new UserRepositoryException(
            "Failed to fetch user",
            e
    );
}
```

The original exception is passed as the cause.

---

## Don't use exceptions for normal control flow

Don't use exceptions where a normal API can handle the situation.

For example:

```
Optional<User>
```

can sometimes be better than throwing an exception for an expected absence.

---

# Part 2 — Generics

## 17. Why Generics?

Without generics:

```
List list =
        new ArrayList();

list.add("Java");
list.add(100);
```

Different types can be inserted.

This can result in runtime errors:

```
String value =
        (String) list.get(1);
```

With generics:

```
List<String> languages =
        new ArrayList<>();
```

Now:

```
languages.add("Java");
```

works.

But:

```
languages.add(100); // ❌
```

Generics provide:

```
Compile-time type safety
+
```

```
Reusable code
+
Less casting
```

---

# 18. Generic Class

```
class Box<T> {

    private T value;

    Box(T value) {
        this.value = value;
    }

    T getValue() {
        return value;
    }
}
```

Usage:

```
Box<String> stringBox =
        new Box<>("Java");

String value =
        stringBox.getValue();
```

Or:

```
Box<Integer> numberBox =
        new Box<>(100);

Integer number =
        numberBox.getValue();
```

Here:

```
Box<T>

T = String
→ Box<String>

T = Integer
→ Box<Integer>
```

```
```

---

# 19. Generic Method

A method can define its own type parameter.

```
public static <T> void printValue(
        T value
) {

    System.out.println(value);
}
```

Usage:

```
printValue("Java");

printValue(100);

printValue(true);
```

Important syntax:

```
<T> void
```

The `<T>` before the return type declares the generic type.

---

# 20. Generic Return Type

```
public static <T> T getFirst(
        List<T> values
) {

    return values.get(0);
}
```

Usage:

```
List<String> names =
        List.of("Ramu", "John");
```

```
String name =
        getFirst(names);
```

Or:

```
List<Integer> numbers =
        List.of(10, 20, 30);

Integer number =
        getFirst(numbers);
```

---

# 21. Multiple Type Parameters

You can have multiple generic types.

```
class Pair<K, V> {

    private K key;
    private V value;

    Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    K getKey() {
        return key;
    }

    V getValue() {
        return value;
    }
}
```

Usage:

```
Pair<String, Integer> employee =
        new Pair<>("Ramu", 101);
```

Mental model:

```
Pair<K, V>
```

```
K → String
V → Integer
```

This is the same idea used by:

```
Map<String, Integer>
```

---

# 22. Bounded Generics

Sometimes a generic type must extend a specific class.

```
public static <T extends Number> double sum(
        T a,
        T b
) {

    return a.doubleValue()
            + b.doubleValue();
}
```

Allowed:

```
sum(10, 20);

sum(10.5, 20.5);
```

Not allowed:

```
sum("10", "20"); // ❌
```

Because:

```
T extends Number
```

means:

```
T must be Number
or a subclass of Number
```

---

# 23. `? extends`

```
List<? extends Number>
```

means:

> A list containing some unknown type that extends `Number`.

For example:

```
List<Integer> numbers =
        List.of(10, 20, 30);

List<? extends Number> values =
        numbers;
```

Or:

```
List<Double> numbers =
        List.of(10.5, 20.5);

List<? extends Number> values =
        numbers;
```

The important rule:

```
? extends
    ↓
Producer
    ↓
READ
```

You can safely read:

```
Number number =
        values.get(0);
```

But you generally cannot add:

```
values.add(10); // ❌
```

because Java doesn't know the exact underlying type.

# 24. `? super`

```
List<? super Integer>
```

means:

> A list whose type is `Integer` or one of its supertypes.

Possible lists:

```
List<Integer>
List<Number>
List<Object>
```

You can safely add:

```
List<? super Integer> numbers =
        new ArrayList<Number>();

numbers.add(10);
numbers.add(20);
```

But when reading:

```
Object value =
        numbers.get(0);
```

The safest type is `Object`.

Mental model:

```
? super
    ↓
Consumer
    ↓
WRITE
```

# 25. PECS

One of the most important Generics rules:

> **PECS = Producer Extends, Consumer Super**

```
Producer
   ↓
? extends

Consumer
   ↓
? super
```

Example:

```
void copyNumbers(
        List<? extends Number> source,
        List<? super Number> destination
) {

    for (Number number : source) {

        destination.add(number);
    }
}
```

Here:

```
source
  ↓
produces Number
  ↓
? extends

destination
  ↓
consumes Number
  ↓
? super
```

---

# 26. Unbounded Wildcard

```
List<?>
```

means:

> A list of an unknown type.

Example:

```
public static void printList(
        List<?> values
) {

    for (Object value : values) {

        System.out.println(value);
    }
}
```

You can pass:

```
List<String>
List<Integer>
List<Employee>
```

etc.

---

# 27. Generic Interface

```
interface Repository<T> {

    void save(T item);

    T findById(int id);
}
```

Implementation:

```
class UserRepository
        implements Repository<String> {

    @Override
    public void save(String item) {

        System.out.println(
                "Saving: " + item
        );
    }
```

```
    @Override
    public String findById(int id) {

        return "User-" + id;
    }
}
```

The implementation decides what `T` is.

```
Repository<T>
      ↓
Repository<String>
```

---

# 28. Practical Generic Repository

Generics become much more useful when building reusable components.

```
class Repository<T> {

    private final List<T> items =
            new ArrayList<>();

    public void save(T item) {

        items.add(item);
    }

    public T findFirst() {

        return items.isEmpty()
                ? null
                : items.get(0);
    }

    public List<T> findAll() {

        return new ArrayList<>(items);
    }
}
```

Usage:

```
Repository<String> userRepository =
        new Repository<>();

userRepository.save("Ramu");
```

```
userRepository.save("Ramu");
userRepository.save("John");

String user =
        userRepository.findFirst();
```

Same repository can be reused:

```
Repository<Integer> repository =
        new Repository<>();

repository.save(10);
repository.save(20);
```

---

# 29. Generic API Response

A very practical backend example:

```
class ApiResponse<T> {

    private boolean success;
    private T data;
    private String message;

    public ApiResponse(
            boolean success,
            T data,
            String message
    ) {
        this.success = success;
        this.data = data;
        this.message = message;
    }

    public boolean isSuccess() {
        return success;
    }

    public T getData() {
        return data;
    }

    public String getMessage() {
        return message;
    }
}
```

Example:

```
record User(
        long id,
        String name
) {}
```

Now:

```
ApiResponse<User> response =
        new ApiResponse<>(
                true,
                new User(1, "Ramu"),
                "User fetched successfully"
        );
```

Here:

```
ApiResponse<T>
       ↓
T = User
```

You could also have:

```
ApiResponse<List<User>>
```

or:

```
ApiResponse<String>
```

This pattern will make much more sense when we build REST APIs in **Level 8/9**.

---

# 30. Generics Cheat Sheet

```
<T>
    Generic type

<T, K>
    Multiple types

<T extends Number>
    Bounded type
```

```
    Bounded type

List<?>
    Unknown type

List<? extends Number>
    Producer / Read

List<? super Integer>
    Consumer / Write

PECS
    Producer Extends
    Consumer Super
```

---

# 31. Exception Cheat Sheet

```
try
    ↓
Code that may fail

catch
    ↓
Handle exception

finally
    ↓
Cleanup

throw
    ↓
Actually throw exception

throws
    ↓
Declare possible exception

Custom Exception
    ↓
Application-specific error

Propagation
    ↓
Exception moves up call stack

try-with-resources
    ↓
Automatically closes resources
```

# 🎯 Level 4 Final Checklist

```
EXCEPTION HANDLING
_____________________________
✅ try
✅ catch
✅ finally
✅ Multiple catch
✅ Multi-catch
✅ Checked Exceptions
✅ Unchecked Exceptions
✅ throw
✅ throws
✅ Custom Exceptions
✅ Exception Propagation
✅ try-with-resources
✅ Exception Best Practices

GENERICS
_____________________________
✅ Why Generics
✅ Generic Classes
✅ Generic Methods
✅ Generic Return Types
✅ Generic Interfaces
✅ Multiple Type Parameters
✅ Bounded Generics
✅ Unbounded Wildcards
✅  ? extends
✅  ? super
✅ PECS
✅ Generic Repository
✅ Generic API Response
```

# 🧠 What You Should Actually Remember

You **don't need to memorize every syntax example**.

The important mental models are:

```
throw
→ actually throw

throws
```

```
→ declare

Checked
→ compiler forces handling/declaring

Unchecked
→ RuntimeException hierarchy

Propagation
→ exception travels up the call stack

try-with-resources
→ automatic resource cleanup
```

And for Generics:

```
<T>
→ reusable type

<T extends X>
→ restrict the type

? extends
→ producer / read

? super
→ consumer / write

PECS
→ Producer Extends
→ Consumer Super
```

---

## Recommended Java notes structure

Your notes folder now becomes:

```
Java/
│
├── Java_Learning_Roadmap.md
│
├── Java_Level_1_Basics_Coding_Challenges.md
│
├── Java_Level_2_Collections_Comparators_Streams.md
│
├── Java_Level_3_OOP_Core_Java.md
│
└── Java_Level_4_Exceptions_Generics.md
```